Design and Implementation of an End-to-End Encrypted Messaging or File Transfer System

COMP 5355 Cyber and Internet Security 2025/2026

Abstract

This project asks you to design, implement, and evaluate an end-to-end encrypted (E2EE) communication system for one of two tasks: one-to-one messaging or file transfer. The implementation must be tied to an explicit, checkable threat model. The system will be assessed on whether it meets the stated security goals, and on whether the report explains, mechanism by mechanism, why those goals hold.

Contents

1 Overview and Motivation

End-to-end encryption has become a standard security feature of modern messaging systems: Signal, WhatsApp, and Apple iMessage all adopt this model to protect message contents from network observers and, to varying degrees, from the service provider itself. The model rests on a simple principle: plaintext should exist only at the communicating endpoints, while servers and the underlying network infrastructure handle nothing more than ciphertext and the metadata required to route it.

The motivation for this end-to-end design is equally straightforward. A provider that can read user messages can be compelled to disclose them, may lose them in a breach, or may reuse them for purposes the user never intended. By keeping plaintext and message keys at the endpoints, the system reduces the trust that must be placed in the server and, in turn, lowers the value of compromising it.

The goal of this project is to understand these core mechanisms by building them yourself. You are not expected to build a production-grade messenger or replicate every feature of a real-world product. Instead, you should build a small, focused system whose security goals are stated clearly and defended carefully: a small system with a precise threat model is preferable to a larger one with unclear guarantees.

The same end-to-end encryption requirement arises in file transfer system, raising the same questions of confidentiality, integrity, and authentication, along with new ones such as handling data too large to protect as a single unit.

2 Task Description

This section defines the two tasks and their functional scope. Each group must choose and implement **one** of the following two tasks. Both share the same threat model (§3.3) and security requirements (§3.4).

2.1 Task 1. End-to-End Encrypted Messaging System.

An End-to-End Encrypted Messaging System is a communication platform that ensures message confidentiality by encrypting data on the sender's device and decrypting it only on the intended recipient's device. Throughout transmission and storage, messages remain encrypted, preventing intermediate servers, network operators, and other third parties from accessing plaintext content. Your message system must support all the following functionalities:

(a) User registration and key-pair generation: each user establishes an identity on the relay server and publishes their public key. The mechanism for authenticating users to the server (e.g. password-based login, pre-shared token) is a design choice; keep it simple, as the focus of this project is the end-to-end protocol, not the client-server authentication.

(b) Authenticated key exchange and session establishment between two users.

(c) Sending and receiving text messages between two online users with the security properties in §3.4.

The following are **optional** features that may be implemented but are not required. If you implement any of them, your report should discuss their security implications under your threat model. Implementing optional features will not earn you bonus scores.

(d) Store-and-forward delivery for offline recipients.

(e) Group messaging.

(f) Media or file attachments within messages.

2.2 Task 2. End-to-End Encrypted File Transfer System.

An End-to-End Encrypted File Transfer System is a secure platform that enables users to exchange files while ensuring that only the intended sender and recipient can access the file contents. Files are encrypted on the sender's device before transmission and remain encrypted during transit, storage, and delivery. The decryption keys are available only to authorized recipients, preventing servers, network operators, and other intermediaries from accessing the plaintext data. Your system must support all the following functionalities:

(a) User registration and key-pair generation: each user establishes an identity on the relay server and publishes their public key. The mechanism for authenticating users to the server (e.g. password-based login, pre-shared token) is a design choice; keep it simple, as the focus of this project is the end-to-end protocol, not the client-server authentication.

(b) Authenticated key exchange and session establishment between two users.

(c) Sending and receiving a file between two users with the security properties in §3.4.

Unlike short messages, files may be too large to encrypt and authenticate as a single unit. If your design divides a file into smaller chunks for encryption or transmission, your report should discuss how the recipient verifies that no chunk has been modified, reordered, duplicated, or omitted, and how it detects that the file is complete.

The following are **optional** features that may be implemented but are not required. If you implement any of them, your report should discuss their security implications under your threat model. Implementing optional features will not earn you bonus scores.

(d) Store-and-forward delivery for offline recipients.

(e) Transfer resumption after interruption.

(f) Concurrent or batch transfer of multiple files.

3 System and Threat Model

This section defines the system architecture, trust assumptions, threat model, and security requirements. Together they form a chain: the system model (§3.1) describes the components and their roles; the trust boundaries (§3.2) state what is assumed trustworthy; the threat model (§3.3) defines the adversaries; and the security requirements (§3.4) state what properties must hold despite those adversaries.

3.1 System Model

The system typically consists of three entities: a sender, a recipient, and a relay server. The sender and recipient each generate and manage their own cryptographic key pairs. Before communication, they exchange public keys via the server and establish session keys through a key agreement protocol. Messages and files are encrypted on the sender's device, transmitted through the relay server, and decrypted on the recipient's device. The relay server provides three core services: user registration,

public-key distribution, and message routing. It may optionally provide store-and-forward delivery for offline recipients (see §2).

3.2 Trust Boundaries and Assumptions

Your design must state what is trusted and what is not. At minimum, address the following points.

- **Endpoints.** The two communicating endpoints are the sole trust anchors: only they hold decryption keys and see plaintext. Each endpoint is assumed to run the protocol correctly and keep all long-term and session secrets confidential.
- **Server.** The relay server is modeled as honest-but-curious: it faithfully executes the protocol but may inspect and log all data it handles. The system design must ensure it learns no plaintext.
- **Network.** All network links are under full adversary control, providing no confidentiality, integrity, or authenticity. All such properties must be established cryptographically by the endpoints.
- **Key Management.** Your design must specify how cryptographic keys are generated, distributed, and verified, and must justify why each step is trustworthy under the declared attacker model. A common approach is to let the relay server store and distribute users' public keys. Under the base threat model the server is honest-but-curious (A3) and will distribute correct keys, but the channel between a client and the server may still be subject to an active network attacker (A2). If your design relies on a transport-layer mechanism such as TLS to protect this channel, state it as an explicit assumption. If your group attempts the malicious-server bonus (A5), the server itself may distribute fake keys, and your design must provide an independent means for users to detect this. Regardless of the distribution method chosen, your report should address key lifecycle decisions such as whether keys are long-lived or ephemeral, how session keys are derived, and what happens when a key is no longer needed.

3.3 Threat Model

The system must address the following adversary classes. Endpoints and their authorized devices are assumed not to be compromised, and standard cryptographic primitives are assumed secure. Availability against an attacker that indefinitely drops or delays traffic is out of scope.

A1. Passive Network Attacker.
Can observe all data in transit, including ciphertext and visible metadata such as timing, sizes, source/destination addresses, and traffic volume. Records everything for later analysis.

A2. Active Network Attacker.
Has all A1 capabilities and may modify, drop, delay, reorder, replay, or inject messages on the wire. This attacker may also attempt a man-in-the-middle attack during connection or key establishment.

A3. Honest-but-Curious Server.
Follows the relay and key-distribution protocol, but has full read access to everything it stores and relays: ciphertext, prekey bundles, and routing metadata. It can record this material for later analysis. It does not modify, drop, reorder, inject, or substitute anything. This case checks whether plaintext stays at the endpoints, in other words whether the server learns no plaintext beyond the ciphertext and the routing metadata required by the protocol.

3.4 Security Requirements

Given the adversaries above, the system must satisfy the following security requirements. In your report, restate each requirement and explain how your design satisfies it.

SR1. Confidentiality. Message bodies and file contents must be accessible only to the intended authorized users. A passive network observer or honest-but-curious relay server must not learn any information about the plaintext beyond the explicitly allowed leakage, such as ciphertext length or padded length.

SR2. Integrity. Any modification, substitution, or forgery of a message or file ciphertext in transit must be detected by the recipient except with negligible probability. The recipient must reject invalid ciphertexts.

SR3. Message and Sender Authenticity. The recipient must be able to verify that an accepted message or file came from the claimed sender and was not forged or injected by a third party.

SR4. Replay Protection. An attacker who captures a legitimate encrypted message must not be able to resend it to the recipient to trigger a duplicate effect.

4 Bonus

This section is optional. It covers two threats that the basic system does not handle: an attacker who reads a device's secret state at one point in time, and a server that actively misbehaves instead of just observing. If your group attempts either, state them in the report and argue it as carefully as the mandatory parts.

4.1 Advanced Attacker Model

These extend the mandatory attackers in §3.3 and are only for the bonus.

A4. Transient Endpoint Compromise.
Reads one device's secret state, including its long-term key, at a single point in time, then loses access. This models a lost device or a one-time key leak. Persistent compromise, where the attacker keeps control of the device while it is in use, is out of scope.

A5. Malicious Server.
Does everything A3 does, but does not have to follow the protocol. It may change, drop, delay, reorder, inject, or withhold relayed data, and it may hand out fake keys during lookup to put itself in the middle.

4.2 Advanced Security Requirements

These continue the numbering in §3.4.

SR5. Forward Secrecy. It is a critical security property in messaging systems ensuring that the compromise of a long-term private key does not compromise the confidentiality of past session keys. In a system that guarantees forward secrecy, if an attacker intercepts and records past encrypted messages, and later manages to steal the user's long-term private keys, the attacker still cannot decrypt these historical encrypted messages.

SR6. Resistance to a Malicious Server. Integrity (SR2) must still hold when the server stops following the protocol. If the relay server changes, drops, reorders, or injects ciphertext, the

recipient rejects it. If the server hands out a fake public key during lookup, the users catch it when they compare fingerprints or safety numbers out of band.

5 Implementation Guidance

This section clarifies what is and is not constrained in the implementation. Unless explicitly stated otherwise, grading will focus on protocol correctness, security properties, interoperability, and clarity of implementation, rather than on the choice of programming language, user interface, or deployment environment.

5.1 Unconstrained Implementation Choices

The following choices are entirely up to your group and have *no* effect on grading.

- **Programming Language.** Any language is acceptable (e.g. Python, Go, Rust, Java, C/C++, JavaScript/TypeScript), provided a vetted cryptographic library (§5.2) is available for it. The client and server need not be written in the same language.
- **User Interface.** A command-line interface, a desktop GUI, or a web front end are all equally acceptable. A minimal CLI that clearly demonstrates the protocol is sufficient for full marks. Polishing the interface earns no additional credit, and time spent on it is better invested in the protocol and the report.
- **Deployment Environment.** The system may run locally on one machine, across multiple processes, in containers, or over a network. A local demo is acceptable if it clearly separates the roles of sender, recipient, and relay server.
- **Transport Protocol.** The relay channel may be implemented using TCP, HTTP, WebSocket or another reasonable communication mechanism. Note that the transport choice does not weaken the threat model: the network is adversarial (§3.2) regardless of the transport.
- **Message Encoding Method.** Message encoding (JSON, Protocol Buffers, a custom binary format, etc.) is a free choice. Whatever you choose, the format of each protocol message should be documented in the report so that the handshake and message-flow diagrams can be checked against the code.

5.2 Cryptographic Guidance

The following rules govern how cryptographic primitives may be used in your implementation.

- **Use a vetted cryptographic library for all low-level primitives.** Do not implement block ciphers, elliptic-curve arithmetic, hash functions, or AEAD modes yourself. Several mature libraries are recommended, including but not limited to `libsodium` (and its bindings such as `PyNaCl`), the Python `cryptography` package, OpenSSL, Go's standard `crypto` packages, and the Web Crypto API.
- **Do not invent cryptographic primitives.** The design work is to compose standard primitives into a protocol. Designing a new cipher or MAC will be treated as a defect.

6 Submission

Each group should submit a single archive (e.g., a `.zip` file) to Learn@PolyU:

1. **Source code** for the client or clients and the relay server, organized and readable, with no secrets committed to the repository. No restrictions on programming languages used.
2. A `README` file with complete build and run instructions: dependencies and versions, how to start the server, how to register users, and a short scripted scenario demonstrating a message exchange or file transfer between two clients.
3. **A group report** in PDF format, which must contain at minimum:
   - A statement of the threat model, including trust boundaries, assumptions, adversary classes, and any out-of-scope settings, adapted to your concrete system.
   - A description of the protocol and key-management lifecycle, with a diagram of the handshake and message flow.
   - A per-adversary defense argument. For each adversary class in the threat model, explain how the design defeats it, name the mechanism involved, and identify the security requirement it satisfies. Where the defense is only partial, state the limitation precisely.
4. **Individual reports** in PDF format describing each group member's individual contribution. Every student should write a separate individual report.

7 Grading Rubric

This section specifies how your project is evaluated. Read each criterion *before* you start coding: every item maps directly to marks. The weight distribution reflects the security-first emphasis of this course, namely 75 % of the base grade depends on threat reasoning, cryptographic correctness, and working defenses.

#	**Criterion**	**Weight**
1	System Design & Threat Model	25 %
2	Cryptographic Correctness	25 %
3	Effective Defense	25 %
4	Code Quality & Documentation	25 %
B1	Forward Secrecy (vs. A4)	+12 %
B2	Malicious-Server Resistance (vs. A5)	+8 %

Table 1: Grading overview. Criteria 1–4 sum to 100 %; bonus extensions are added on top.

7.1 Criterion 1: System Design & Threat Model (25 %)

- The system design should be clearly presented, with detailed descriptions, architecture diagrams, and message-flow diagrams that match the submitted implementation.
- A clearly scoped threat model: trust boundaries, system assumptions, and what is explicitly *out of scope*.
- For each adversary (A1–A3): which defense mechanism is used (e.g. AEAD for integrity against A2), which security requirement (SR1–SR4) it satisfies, and any residual limitations.

7.2 Criterion 2: Cryptographic Correctness (25 %)

- All primitives should come from a vetted library (e.g. `libsodium`, `OpenSSL`).
- Key management should be sound. For example, keys are generated with a cryptographic random number generator, nonces and counters are never reused.
- No secrets (keys, passwords, tokens) appear in network traffic in plaintext.

7.3 Criterion 3: Effective Defense (25 %)

- The running system actually resists the adversaries declared in Criterion 1, which could be verified through demo in presentation, security analysis in report, and code review.
- Do not overstate what the implemented system can actually defend.

7.4 Criterion 4: Code Quality & Documentation (25 %)

- Code is organized into logical modules with consistent naming and proper comments.
- The repository contains no committed secrets, credentials, or private keys.
- The `README` includes complete instructions (e.g., dependencies, build commands).
- The demonstration shows a working message exchange or file transfer between two clients.
- The group report documents the protocol completely and accurately, so that the implementation can be checked against the design.

7.5 Bonus Extensions (up to +20 %)

A group may attempt one or both of the following extensions from §4. Each is graded independently.

B1. Forward Secrecy vs. A4 (+12 %)
Resist a key-compromise adversary: compromise of long-term keys must not decrypt past sessions. The higher weight reflects the additional implementation effort typically required for an ephemeral key-exchange protocol.

B2. Malicious-Server Resistance vs. A5 (+8 %)
Resist an untrusted relay, namely the malicious server cannot undetectably tamper with messages.

Both extensions are held to the same standard as the base criteria: state the threat, name the mechanism, and argue why it works. Partial credit is available for each extension independently—an incomplete implementation with a sound security argument still earns marks.